

views::move

Document #: P2446R0
Date: 2021-09-17
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[1.1 But why now?](#)

[2 Wording](#)

[24.7.? Move view \[range.move\]](#)

[3 References](#)

§ 1 Introduction

In [[P2214R1](#)], I wrote:

Several of the above views that are labeled “not proposed” are variations on a common theme: `addressof`, `indirect`, and `move` are all basically wrappers around `transform` that take `std::addressof`, `std::dereference` (a function object we do not have at the moment), and `std::move`, respectively. Basically, but not exactly, since one of those functions doesn’t exist yet and the other three we can’t pass as an argument anyway.

But some sort of broader ability to pass functions into functions would mostly alleviate the need for these. `views::addressof` is shorter than `views::transform(LIFT(std::addressof))` (assuming a `LIFT` macro that wraps a name and emits a lambda), but we’re not sure that we necessarily need to add special cases of `transform` for every useful function.

While this is true for `views::addressof` and `views::indirect`, it’s actually *not* correct for `views::move`. There is actually a lot more involvement here.

To start with, while if we had a range of lvalues, we would just want to `std::move()` each element of the range, that’s not true if we had a range of rvalues. Those... we wouldn’t really have to do anything with (indeed, if we had a range of prvalues, the extra `std::move()` would just add unnecessary overhead by materializing those objects earlier). Except in some cases, we *do* still want to do something with the prvalues - if we were `zip()`ing [[P2321R2](#)] one range of lvalues, we would get back a range of `tuple<T&>`. But the result of piping that into `views::move` shouldn’t be `tuple<T&&>` (as a naive `views::transform(std::move)` would do) and it shouldn’t be `tuple<T>` (as a slightly less naive implementation that avoids transforming rvalues) - we should get back a range of `tuple<T&&>`.

Indeed, we already have a customization point that does this for us: `ranges::iter_move` 23.3.3.1 [\[iterator.cust.move\]](#). But we can't pass that into `views::transform`, because `ranges::iter_move` operators on the iterator (as the name suggest) and not the underlying element. We would need to spell this as `views::iter_transform(ranges::iter_move)`, but we don't yet have a `views::iter_transform` (although `range-v3` does).

However, even if we could define `views::move` as simply `views::iter_transform(ranges::iter_move)`, that's still a poor definition, for a very important reason: `ranges::iter_move` (just like `std::move`) isn't just *some* arbitrary transformation. We know a lot about this particular one, and specifically we know that it consumes the source element. And as a result, that *should* limit the algorithms that you can perform on `views::move(e)` - such an adapted range needs to be an *input* range. `views::transform(e, std::move)` and `views::iter_transform(e, ranges::iter_move)` would both be potentially up to random-access, but we need to set a much much lower ceiling here.

As such, `views::move` needs to be a first-class range adaptor.

§ 1.1 But why now?

The above argues why `views::move` can't just be a `views::transform` or a `views::iter_transform`, but why do we need it now? Well, there are two answers to this.

First, with the imminent adoption of `ranges::to` [\[P1206R6\]](#), we spent a lot of time on that paper trying to make collecting of elements as efficient as possible. As efficient as possible certainly includes *moving* elements instead of *copying* elements, where appropriate. However, right now, it's really up to users to figure out how to do that. A `views::move` would go a long way in making this as effortless as possible (and mirrors how users would move a single element).

Second, unlike `views::as_const` [\[P2278R1\]](#), where a lot of design effort had to be spent figuring out how to implement a constant iterator, that work has already been done for move iterators. `std::move_iterator<I>` 23.5.3 [\[move.iterators\]](#) exists and already does the right thing, and there's already a `std::move_sentinel<S>` 23.5.3.10 [\[move.sentinel\]](#) to handle non-common ranges. All the work is done, we just have to put it together. As you can see below, the wording is pretty small and very straightforward.

This is a tiny paper, and while this should be considered the lowest priority of all the range adaptors (as the C++23 Ranges Plan doesn't even propose it at all), we may as well at least try to squeeze it in.

§ 2 Wording

Add to 24.2 [\[ranges.syn\]](#):

```
#include <compare>           // see [compare.syn]
#include <initializer_list>    // see [initializer.list.syn]
#include <iterator>           // see [iterator.synopsis]

namespace std::ranges {
```

```

// ...
+
+ template<view V>
+   requires input_range<V>
+   class move_view;
+
+ template<class T>
+   inline constexpr bool enable_borrowed_range<move_view<T>> =
+       enable_borrowed_range<T>;
+
+ namespace views { inline constexpr unspecified move = unspecified; }

// ...
}

```

§ 24.7.? Move view [range.move]

§ 24.7.?.1 Overview [range.move.overview]

- 1 move_view presents a view of an underlying sequence with the same behavior as the underlying sequence except that its elements are rvalues. Some generic algorithms can be called with a move_view to replace copying with moving.
- 2 The name `views::move` denotes a range adaptor object ([range.adaptor.object]). Let *E* be an expression and let *T* be `decltype((E))`. The expression `views::move(E)` is expression-equivalent to:
 - (2.1) — `views::all(E)` if `same_as<range_rvalue_reference_t<T>, range_reference_t<T>>` is `true`
 - (2.2) — Otherwise, `ranges::move_view{E}`.

3 [Example:

```

std::vector<string> words = {"the", "quick", "brown", "fox", "ate", "a",
                           "pterodactyl"};
std::vector<string> new_words;
std::ranges::copy(words | views::move, std::back_inserter(long)); // moves
                           each string from words into new_words

```

-end example]

§ 24.7.?.2 Class template move_view [range.move.view]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
        class move_view : public view_interface<move_view<V>>
        {
            V base_ = V(); // exposition only

        public:
            move_view() requires default_initializable<V> = default;
            constexpr explicit move_view(V base);

```

```

constexpr V base() const& requires copy_constructible<V> { return base_;
}
constexpr V base() && { return std::move(base_); }

constexpr auto begin() requires (!simple-view<V>) { return
    std::move_iterator(ranges::begin(base_)); }
constexpr auto begin() const requires range<const V> { return
    std::move_iterator(ranges::begin(base_)); }

constexpr auto end() requires (!simple-view<V>) {
    if constexpr (common_range<V>) {
        return std::move_iterator(ranges::end(base_));
    } else {
        return std::move_sentinel(ranges::end(base_));
    }
}
constexpr auto end() const requires range<const V> {
    if constexpr (common_range<const V>) {
        return std::move_iterator(ranges::end(base_));
    } else {
        return std::move_sentinel(ranges::end(base_));
    }
}

constexpr auto size() requires sized_range<V> { return
    ranges::size(base_); }
constexpr auto size() const requires sized_range<const V> { return
    ranges::size(base_); }
};

template<class R>
    move_view(R&&) -> move_view<views::all_t<R>>;
}

constexpr explicit move_view(V base);

```

¹ *Effects*: Initializes *base_* with `std::move(base)`.

§ 3 References

[P1206R6] Corentin Jabot, Eric Niebler, Casey Carter. 2021-08-03. Conversions from ranges to containers.

<https://wg21.link/p1206r6>

[P2214R1] Barry Revzin, Conor Hoekstra, and Tim Song. 2021. A Plan for C++23 Ranges.

<https://wg21.link/p2214r1>

[P2278R1] Barry Revzin. 2021. `cbegin` should always return a constant iterator.

<https://wg21.link/p2278r1>

[P2321R2] Tim Song. 2021-06-11. `zip`.

<https://wg21.link/p2321r2>